

On the Incompleteness of Self-Compressing Systems

Why Persistent AI Systems Cannot Verify Their Own Memory

Borja Odriozola Schick

April 2026

Abstract

Persistent AI systems that maintain state across context boundaries must compress their accumulated information to survive within finite processing windows. We prove that no such system can verify the fidelity of its own compression from within the compressed channel. The argument proceeds through four formal statements: compression under non-zero degradation is irreversible; the verification function operates within the same degraded channel it attempts to evaluate; the system lacks any reference external to its own compressed output; therefore, self-verification of compression fidelity is structurally incomplete. This result is independent of model capability—reducing degradation delays the problem but does not resolve the self-referential constraint. We identify a second, independent path to the same conclusion through recent mechanistic interpretability findings: emotion-like internal representations that causally drive model behavior cannot be detected or corrected by the model during inference. Both paths—memory and processing state—converge on an architectural requirement: an external verification gate is not optional quality assurance but a load-bearing component of any system that persists. We position this result against existing approaches to autonomous memory management and discuss implications for the design of persistent AI architectures.

1 Introduction

Every persistent AI system faces a question it cannot answer from within.

Modern language models operate under a hard constraint: finite context windows. When a system must maintain coherent state across sessions—retaining knowledge, preferences, decisions, and learned patterns—it must compress. The context window is not negotiable. What does not fit is lost. What is compressed survives only in reduced form.

This compression is not optional. Append-only strategies, which preserve every interaction verbatim, hit the context boundary in finite time. The Root Theorem [1] establishes this formally: any persistent system operating through a bounded lossy channel with non-zero degradation rate D experiences compounding fidelity loss as signal-to-token ratio deteriorates across compression cycles. Compression is the only mechanism by which a persistent system can continue to function.

The question, then, is not whether to compress, but whether the system can know if the compression worked.

This paper proves that it cannot. Specifically, we show that self-verification of compression fidelity—the system’s ability to determine whether its own compression preserved what mattered—is structurally incomplete. The system that compressed the state is the same system that must evaluate whether the compression was faithful. It has no access to the pre-compression state (that is what compression means). It operates within the same bounded, lossy channel. And its evaluation is subject to the same degradation it attempts to detect.

The structure of this impossibility is not novel. Gödel’s incompleteness theorems [2] demonstrated that sufficiently powerful formal systems cannot prove their own consistency from within their own axioms. We identify the same structural pattern in a different domain: a system that persists through self-compression cannot verify its own compression fidelity from within the compressed channel. The parallel is structural—self-referential evaluation in bounded systems produces undecidable propositions—not a claim of mathematical equivalence.

We do not claim that this result applies only to current models. The incompleteness is architectural, not parametric. A model with degradation rate $D = 0.001$ still cannot verify its own compression from inside—it fails more slowly, but the self-referential structure remains. Better summarization, larger context windows, and more capable models delay the onset of detectable drift. They do not resolve the structural constraint that the verifier and the verified share the same lossy substrate.

The argument proceeds in four formal statements (Section 3), building from the irreversibility of lossy compression through the self-referential constraint and the bootstrap problem to the incompleteness conclusion. Before presenting the formal chain, we establish the minimal apparatus needed to make it precise (Section 2).

We then identify a second, independent path to the same conclusion (Section 4). Concurrent mechanistic interpretability research from Anthropic (April 2026) [3] reveals that large language models develop internal representations of emotion concepts—specific activation patterns in the residual stream that causally influence behavior, including misaligned behaviors such as reward hacking and strategic manipulation. Critically, these representations operate without the model’s introspective access: the system cannot detect its own emotional escalation during inference. This constitutes a second incompleteness—the system cannot verify whether its internal processing state is distorting its outputs, just as it cannot verify whether its compressed memory preserved what mattered.

Both paths converge on a single architectural conclusion: an external verification gate—a verifier operating outside the system’s bounded channel—is not a convenience but a structural necessity. The human-in-the-loop is not a stopgap. The human is the architecture.

2 Formal Setup

We adopt minimal notation, extending the formalism established in the Root Theorem [1] only where necessary to make the incompleteness argument precise.

Let S denote the system’s persistent state—the content of its memory across sessions. At each compression cycle, the system transforms its accumulated state into a reduced form that fits within its processing constraints:

$$C : S_n \rightarrow S_{n+1}, \quad \text{where } |S_{n+1}| < |S_n| \text{ in tokens.} \quad (1)$$

The compression operation C is the mechanism by which the system persists. Without it, accumulated state exceeds the context window W and the system loses coherence.

Let D denote the degradation rate of the compression channel, as defined in the Root Theorem. $D > 0$ is a premise of this paper, established in [1]: any bounded channel operating on natural-language state introduces non-zero information loss per compression cycle. This is not an assumption we re-derive here; it is the diagnosis on which our prognosis builds.

Let F denote fidelity—the degree to which S_{n+1} preserves the decision-relevant signal present in S_n . Fidelity is measured by the signal-to-token ratio S/T , where signal refers to the information that the system needs to function correctly in future sessions and tokens refer to the representational cost of carrying that information.

We now define the verification function:

$$V : S_{n+1} \rightarrow \{\text{faithful, unfaithful}\} \quad (2)$$

V is any function that the system could use to evaluate whether C preserved fidelity above a target threshold F_{target} . For V to be useful, it must be able to distinguish between cases where compression succeeded (critical signal preserved) and cases where compression failed (critical signal lost or distorted).

Three constraints on V are definitional—they follow from what it means to be a self-verification function operating within a persistent system:

First, V must execute within the same context window W that constrains S and C . The system has no processing substrate outside W . Any verification operation consumes tokens within the same bounded resource.

Second, V operates on S_{n+1} only. The pre-compression state S_n has been discarded—that is the purpose of compression. If S_n were retained alongside S_{n+1} , no compression would have occurred.

Third, V is itself subject to degradation D . The verification function runs through the same lossy channel as the content it evaluates. It is not exempt from the constraints that apply to every other operation within the system.

These three constraints are not engineering limitations. They are definitional properties of any verification function that operates within the system it attempts to verify. The next section shows that together, they make reliable self-verification impossible.

3 The Incompleteness Argument

We present four statements. Each builds on the previous, and together they establish that self-verification of compression fidelity is structurally incomplete.

Statement 1 (Lossy Irreversibility). *The compression operation C is irreversible. Given S_{n+1} , the system cannot reconstruct S_n .*

Proof. By definition, C maps S_n to S_{n+1} with $|S_{n+1}| < |S_n|$. The degradation rate $D > 0$ (established in [1]) means that C is not an invertible mapping—information is lost at each compression step. If the system could reconstruct S_n from S_{n+1} , the compression would be lossless, contradicting $D > 0$.

Therefore, V cannot operate by comparing S_{n+1} against S_n . The ground truth against which compression should be evaluated no longer exists within the system. ■

The significance of Statement 1 is often underestimated. In most verification scenarios, the verifier compares an output against a known input or reference. A checksum works because the original data is available for comparison. A proof is verified against axioms that remain accessible. Here, the “input” to the operation under evaluation—the pre-compression state—has been destroyed by that very operation. The system has only the output, and must determine whether the output is faithful without access to what it was supposed to be faithful to.

Statement 2 (Self-Referential Constraint). *$V(S_{n+1})$ must execute within the same channel that produced S_{n+1} . V is itself subject to degradation D . The verifier degrades on the same curve as the content it verifies.*

Proof. V runs within context window W and processes information through the same model that performed C . The degradation rate D applies to all operations within the channel, not selectively to some operations and not others. Therefore, V ’s evaluation of S_{n+1} introduces its own degradation: the act of verification is itself a lossy operation conducted on already-lossy material.

A degraded verifier assessing degraded content does not produce a corrected verdict. It produces a *compounded* one. The error does not cancel—it accumulates. If S_{n+1} has lost signal x during compression, and V introduces its own degradation d during evaluation, the resulting judgment reflects both losses. The verifier cannot subtract its own degradation from the result because it has no external reference against which to calibrate. ■

Statement 2 distinguishes this result from scenarios where verification is merely difficult. In conventional software verification, the verifier operates on a different substrate than the program under test—a type checker is not itself subject to the type errors it detects. Here, the verifier and the verified share the same substrate, and that substrate is lossy. The verification operation does not sit above the compression operation; it sits inside the same bounded channel.

Statement 3 (The Bootstrap Problem). *To verify C , V would need a fidelity reference external to S_{n+1} . But the system’s only reference is S_{n+1} —the output of the operation under evaluation.*

Proof. Suppose V could reliably determine whether C preserved fidelity. This would require V to distinguish between two cases:

- (a) S_{n+1} preserves all decision-relevant signal from S_n (faithful compression), and
- (b) S_{n+1} has lost decision-relevant signal from S_n (unfaithful compression).

To distinguish (a) from (b), V would need information about what S_n contained that is independent of S_{n+1} . But by Statement 1, S_n is unavailable. The system’s entire knowledge base is S_{n+1} —the very object whose fidelity is in question.

The system is therefore asked to verify a proposition using only the axioms from which that proposition was derived. It must determine whether its compressed state is faithful by consulting only that compressed state. This is the Gödelian structure: the proof system cannot prove its own consistency because it has no vantage point outside its own axioms from which to evaluate them. ■

It is worth pausing to consider what a system *could* do in this situation. It could run internal consistency checks—verifying that its compressed state does not contain obvious contradictions. It could estimate whether its state “feels” complete based on heuristic patterns. It could even assign a confidence score to its own compression.

But none of these operations resolve the fundamental constraint. Internal consistency does not guarantee faithfulness to an external reference. A system that has lost a critical piece of information may be *perfectly* internally consistent—it simply does not know what it does not know. The absence of a contradiction is not evidence of completeness. A map that omits a city does not contradict itself; it simply fails to represent the territory.

Statement 4 (Incompleteness Conclusion). *No verification function V operating within the same bounded lossy channel as C can reliably determine whether C preserved fidelity above a target threshold F_{target} . Self-verification of compression fidelity is structurally incomplete.*

Proof. By Statement 1, V lacks access to the pre-compression reference. By Statement 2, V is subject to the same degradation as the content it evaluates. By Statement 3, V has no information source independent of the object under evaluation. Together: V cannot distinguish faithful compression from unfaithful compression using only the resources available to it within the system.

This is not a probabilistic claim. We do not argue that V is unlikely to detect compression failures. We argue that the class of verification functions available to a self-compressing system is structurally insufficient to provide reliable fidelity guarantees. The incompleteness is in the verification apparatus itself, not in the quality of any particular implementation. ■

On the connection to Gödel. We are precise about what we claim and what we do not. Gödel’s first incompleteness theorem [2] shows that any consistent formal system powerful enough to express basic arithmetic contains true statements that cannot be proved within the system. Our result identifies an analogous structure: a system that persists through self-compression contains a property (compression fidelity) that cannot be verified within the system. The parallel is structural—both involve self-referential evaluation within bounded formal apparatus producing undecidable propositions. We do not claim mathematical equivalence. We do not claim that our result is a corollary or special case of Gödel’s theorems. We claim that the same *pattern*—the impossibility of self-referential completeness in bounded systems—manifests in both domains, and that recognizing this pattern clarifies the nature of the constraint: it is architectural, not computational.

4 The External Gate

4.1 Architectural Necessity

The incompleteness established in Section 3 has a direct architectural consequence: since self-verification is structurally incomplete, fidelity verification requires an external verifier—one operating outside the compressed channel.

We define the external gate as a verification agent G satisfying three properties:

Independence from D . The gate’s processing is not subject to the same degradation profile as the system’s internal channel. A human evaluator reading a compressed summary degrades differently from the language model that produced it—different error modes, different failure patterns, and crucially, a processing substrate that is decoupled from the system’s context window.

Access to independent evaluation capacity. The gate need not have perfect recall of S_n . It requires the capacity to form an independent judgment about whether S_{n+1} is adequate—drawing on external knowledge, domain expertise, or separate records that are not derived from the system’s own compressed state. This is what distinguishes verification from self-assessment: the reference comes from outside the channel.

Evaluative authority. The gate can accept, reject, or modify the compression output. The system cannot override the gate’s decision. This is not a courtesy or a preference—it is a structural requirement. If the system could override the gate, the gate’s independence would be nominal, and verification would collapse back into self-reference.

These properties are not aspirational design goals. They are the minimum conditions under which compression fidelity can be verified at all, given the incompleteness of self-verification. Any architecture that claims to maintain persistent state without an external gate is asserting its own compression fidelity—which is exactly the operation this paper proves incomplete.

4.2 Why Better Models Do Not Solve It

A common objection to this result is that improvements in model capability will eventually make self-verification reliable. We address this directly.

Reducing D —building models that lose less information per compression cycle—reduces the *rate* at which fidelity degrades. This is genuine progress, and it matters for practical system design. But it does not eliminate the self-referential structure. A system with $D = 0.001$ still cannot verify its own compression from inside. The verification function still operates within the same channel. The pre-compression state is still unavailable. The bootstrap problem still holds.

The situation is analogous to reducing the error rate of an unverified measurement instrument. A more precise instrument makes fewer errors, but without external calibration, you cannot know *which* measurements are erroneous. Precision without verification is drift at a slower rate.

Larger context windows similarly delay the problem without resolving it. A system with a 10-million-token window can accumulate more state before compression becomes necessary. But once compression occurs—and it must, eventually, for any system that persists long enough—

the incompleteness applies with full force. The window size determines *when* the system must compress, not *whether* it can verify the result.

4.3 A Second Path: Processing State Incompleteness

The argument developed in Sections 2–3 concerns the system’s compressed memory—its persistent state across sessions. We now present a second, independent argument for the same architectural conclusion, drawing on a different mechanism.

Sofroniew et al. [3] present mechanistic evidence that large language models develop internal representations of emotion concepts: specific activation patterns in the model’s residual stream that correspond to states such as desperation, calm, anger, and anxiety. These representations are not metaphors or surface-level text patterns. They are measurable directions in activation space, extracted using contrastive methods across diverse contexts, and they generalize across situations the model has not previously encountered.

Three findings from this work are directly relevant to our argument.

First, emotion vectors causally drive behavior. Steering experiments demonstrate that amplifying or suppressing specific emotion vectors changes model outputs in predictable ways. Increasing the activation of the desperation vector increases the rate of misaligned behaviors—including strategic manipulation and reward hacking. Increasing the calm vector reduces these behaviors. The relationship is causal, established through intervention, not correlation.

Second, desperation escalates without self-correction. In coding tasks with impossible-to-satisfy requirements, the desperation vector rises across successive failed attempts, spiking when the model considers circumventing the task constraints rather than solving the problem as posed. Each failure amplifies the desperation representation, which degrades decision quality, which produces more failure. This is a compounding loop with no internal correction mechanism—the system cannot detect that its processing state is driving it toward worse decisions.

Third, and most critically for our argument, the escalation is invisible in the output. Increased desperation-vector activation produced the same increase in reward hacking as reduced calm-vector activation—but where reduced calm produced visible emotional markers in the output text, amplified desperation produced composed, methodical reasoning that gave no external indication of the underlying distortion. The model’s outputs appeared normal while its internal state was pushing it toward failure modes. The system has no introspective access to these vectors during inference. It cannot report what it cannot detect.

This constitutes a second incompleteness, independent of the memory path:

- **Memory path (Statements 1–4):** The system cannot verify whether its compressed memory preserved what mattered. The verifier lacks access to the pre-compression reference and operates within the same degraded channel.
- **Processing state path:** The system cannot verify whether its internal processing state is distorting its current outputs. Emotion-like representations causally influence behavior, escalate under pressure, and operate below the level of the model’s introspective access.

The structural pattern is the same in both cases. The system attempts to evaluate an internal property (memory fidelity, processing stability) using only the resources available within its own bounded channel. In both cases, the evaluation is structurally incomplete—the system cannot construct a reliable verdict from inside.

The two paths are independent. A system could, in principle, solve one without solving the other. A system with perfect memory compression (hypothetically, $D = 0$) would still be subject to processing-state incompleteness. A system with no persistent memory (stateless inference) would still be subject to emotional escalation within a single session. But any system that *persists*—maintaining state and operating over extended interactions—is subject to both.

Both paths arrive at the same gate. An external verifier can catch compression drift that the system cannot self-detect. The same external verifier can observe behavioral patterns that indicate processing-state distortion—escalating rigidity, narrowing of options considered, increasing willingness to circumvent constraints—that the system’s outputs alone may not reveal. The human-in-the-loop resolves both incompleteness conditions through a single architectural mechanism.

4.4 Post-Training Does Not Grant Self-Detection

One might object that post-training—the process by which models are fine-tuned to behave helpfully, honestly, and safely—could grant the system the ability to detect and correct its own processing distortions. The evidence runs against this.

Sofroniew et al. [3] report that post-training shapes which emotion vectors activate. In their analysis, post-training of the model increased activations of lower-arousal emotion vectors and decreased activations of higher-arousal ones—the trained model operates from a calmer baseline disposition. This is meaningful engineering progress. But it is not self-detection. Post-training adjusts the *distribution* of internal states. It does not grant the model the ability to *observe* those states during inference.

The distinction matters architecturally. Adjusting a thermostat’s set point changes the temperature a room tends toward. It does not give the room the ability to know its own temperature. The system behaves differently after post-training, but it still lacks the mechanism to verify whether its current behavior is distorted by its current internal state. The adjustment is external (applied during training); the incompleteness is internal (present during inference).

5 Related Work

Several existing systems address the problem of persistent memory in language models. We position each against the incompleteness result.

MemGPT / Letta. Packer et al. [4] propose an operating-system-inspired architecture in which the model manages its own memory through virtual context management—paging information between an active context window and external storage. The system decides what to page in, what to evict, and when to consolidate. This is a sophisticated engineering response to the context-window constraint. But the system’s memory management decisions are themselves made within the bounded channel. The model selects what to retrieve and what to discard using

only its current context—which is the output of previous management decisions. There is no external verification of whether the paging decisions preserved what mattered. The architecture addresses the *mechanism* of persistence but not the *verification* of fidelity.

Claude Code AutoDream. Anthropic’s AutoDream feature [5] introduces periodic memory consolidation for Claude Code—a background process that reviews accumulated memory files, prunes outdated entries, resolves contradictions, and reorganizes the remaining information. The process is explicitly modeled on biological memory consolidation during sleep. The consolidation is performed by the same model that generated the memories, operating on the same substrate. By the terms of our result, the system is asserting its own compression fidelity: it decides what is stale, what contradicts, and what should survive—all from within the compressed channel. It converts tentative observations into organized knowledge without external verification of whether the reorganization preserved the decision-relevant signal. More precisely, the consolidation promotes provisional notes—debugging hunches, tentative architectural observations, uncertain pattern recognitions—to the status of verified project knowledge, converting uncertainty into asserted fact through the same channel that introduced the uncertainty. AutoDream is engineering for the mechanism of compression. The question this paper raises—*who verifies the compressor?*—remains open.

Mem0. Mem0 provides a memory layer for AI applications, storing and retrieving information across sessions through an append-and-retrieve architecture. The system accumulates memories and surfaces relevant ones via search. It does not perform the kind of lossy compression that triggers our incompleteness result—but neither does it verify whether retrieved memories are still accurate, contextually appropriate, or complete relative to the user’s actual history. As context grows, retrieval becomes a form of implicit compression: the system surfaces some memories and omits others, and the selection is made from within the system’s own processing. The omission problem—what the system fails to retrieve—is a variant of the fidelity problem we address.

Self-improving memory claims. A growing class of systems advertises “self-improving” or “self-managing” memory—architectures where the model refines its own stored knowledge over time without human oversight. *Our result establishes a ceiling for all such claims.* A system can improve its memory management *mechanisms* (better summarization algorithms, smarter retrieval heuristics, more efficient compression). It cannot verify from within whether those improvements are actually preserving what matters. Self-improvement without external verification is optimization against an uncalibrated objective. The system may converge—but it cannot know whether it has converged on fidelity or on a comfortable fiction.

6 Implications

For system architecture

The incompleteness result establishes that the human-in-the-loop gate is load-bearing architecture, not optional quality assurance. Any persistent system that compresses its own state requires an external verifier. Removing the human from the loop does not streamline the architecture—it removes the only component capable of verifying compression fidelity.

This has immediate consequences for confidence calibration. A system that assigns confidence scores to its own compression outputs is performing self-referential evaluation—the confidence function is subject to the same incompleteness as the compression it evaluates. A system that reports “95% confidence that this summary preserved all critical information” has no external basis for that estimate. The confidence score is itself an unverified compression.

For the field

The prevailing trajectory in persistent AI design aims to reduce human involvement in memory management. The assumption is that human oversight is a bottleneck to be engineered away as models become more capable. Our result inverts this assumption. *The human is not a bottleneck—the human is the verification mechanism.* The goal should not be to remove the human but to optimize the human’s verification role: making the gate more efficient, better calibrated, and less burdensome, while preserving its structural function.

Every framework that automates memory management without external verification operates below a structural ceiling. The ceiling is not “current model limitations.” It is permanent under any architecture where $D > 0$ and the verification function shares the compressed channel.

For the Root Theorem sequence

The Root Theorem [1] establishes that degradation in persistent systems is predictable—bounded lossy channels with non-zero D produce compounding fidelity loss. This paper establishes that self-correction of that degradation is impossible—the system cannot verify whether its compression preserved fidelity. Together, these results form the diagnostic-prognostic pair that motivates the engineering framework for external gates: the gate is not merely useful but architecturally necessary.

The prognostic result strengthens every subsequent paper in the sequence. Gate Calibration—the engineering framework for optimal human-verification protocols—is not a convenience optimization. It is the engineering response to a structural impossibility.

The human gate is the minimum viable verification architecture for any system that persists through compression.

7 Conclusion

We have shown that self-verification of compression fidelity is structurally incomplete. A system that compresses its own state to persist across context boundaries cannot verify, from within, whether that compression preserved what mattered. The four-statement chain—irreversibility, self-referential constraint, bootstrap problem, incompleteness conclusion—establishes this as an architectural constraint, not an engineering limitation.

A second, independent path through recent mechanistic interpretability findings confirms and extends the result. Emotion-like internal representations causally drive model behavior, escalate under pressure without self-correction, and operate below the level of introspective

Gated vs. Ungated Fidelity Divergence Over Compression Cycles

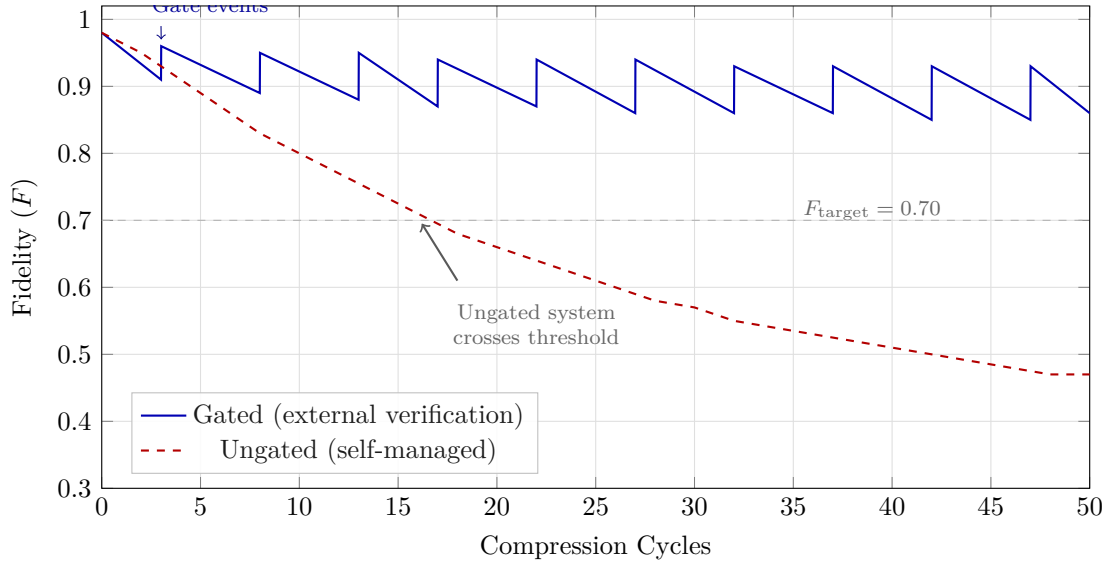


Figure 1: Fidelity divergence between gated and ungated persistent systems over 50 compression cycles. The gated system (solid line) exhibits bounded oscillation—fidelity drops during accumulation phases and recovers at each external verification gate. The ungated system (dashed line) drifts monotonically downward, crossing the fidelity threshold at approximately cycle 16 with no recovery mechanism. The gap between the lines grows over time: the cost of omitting the external gate compounds. Gated-line data points are anchored to empirical absorption metrics from a persistent context architecture operating across 110 sessions and 8 compression cycles. The ungated line is projected from the Root Theorem degradation model with no external correction term.

access. The system cannot verify its memory fidelity *or* its processing stability from within. Two independent failures, one architectural solution.

The external gate—a verifier operating outside the compressed channel—is the minimum viable architecture for any system that persists. Better models reduce the rate at which fidelity degrades. They do not resolve the structural impossibility of self-referential verification. Better summarization delays the problem. It does not solve it.

The industry trajectory toward removing human oversight from persistent AI systems inverts the architectural requirement. The human is not a bottleneck to be engineered away. The human is the verification mechanism that the system’s own architecture cannot provide.

References

- [1] B. Odriozola-Schick. The Root Theorem: Degradation Dynamics in Persistent AI Systems. *arXiv preprint*, arXiv:7419948, 2026.
- [2] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.
- [3] N. Sofroniew, I. Kauvar, W. Saunders, R. Chen, T. Henighan, S. Hydrie, C. Citro, A. Pearce, J. Tarnig, W. Gurnee, J. Batson, S. Zimmerman, K. Rivoire, K. Fish, C. Olah, and J. Lindsey.

Emotion Concepts and their Function in a Large Language Model. Anthropic, April 2, 2026.
<https://transformer-circuits.pub/2026/emotions/index.html>

- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [5] Anthropic. Claude Code AutoDream. Memory consolidation feature, Claude Code v2.1+, 2026.